

Lab8

Database Handling in Flutter

Aim

To understand database handling in Flutter and build applications that can store, retrieve, update, and delete structured data persistently using SQLite.

Learning Outcomes

- Understand why in-memory data is lost when an application terminates.
- Understand persistent data and application state.
- Understand tables, rows, columns, primary keys, and SQLite data types.
- Add and use the sqflite and path packages.
- Create and open a SQLite database and create tables.
- Perform INSERT, SELECT, UPDATE, and DELETE operations.
- Use parameterized WHERE clauses.
- Convert Dart model objects to database maps and database rows back to model objects.
- Build a reusable database helper class.
- Connect asynchronous database operations with Flutter UI state.
- Understand transactions, batches, and database migration.
- Convert the Lab 7 Task Manager from in-memory CRUD to persistent SQLite CRUD.

SQLite and Need for Persistence

What is SQLite?

SQLite is an **embedded relational database** in which the database is stored as a local file and accessed directly by the application. It does not require a separate database server.

Why is Persistence Needed?

In Lab 7, task data was stored in an in-memory `List<Task>`. When the application was closed, the memory was released and the tasks were lost.

In Lab 8, the tasks are stored in SQLite. Therefore, the data remains available even after closing and reopening the application.

Flow:

Add Task → INSERT into SQLite → Reload Data → setState() → UI

After closing:

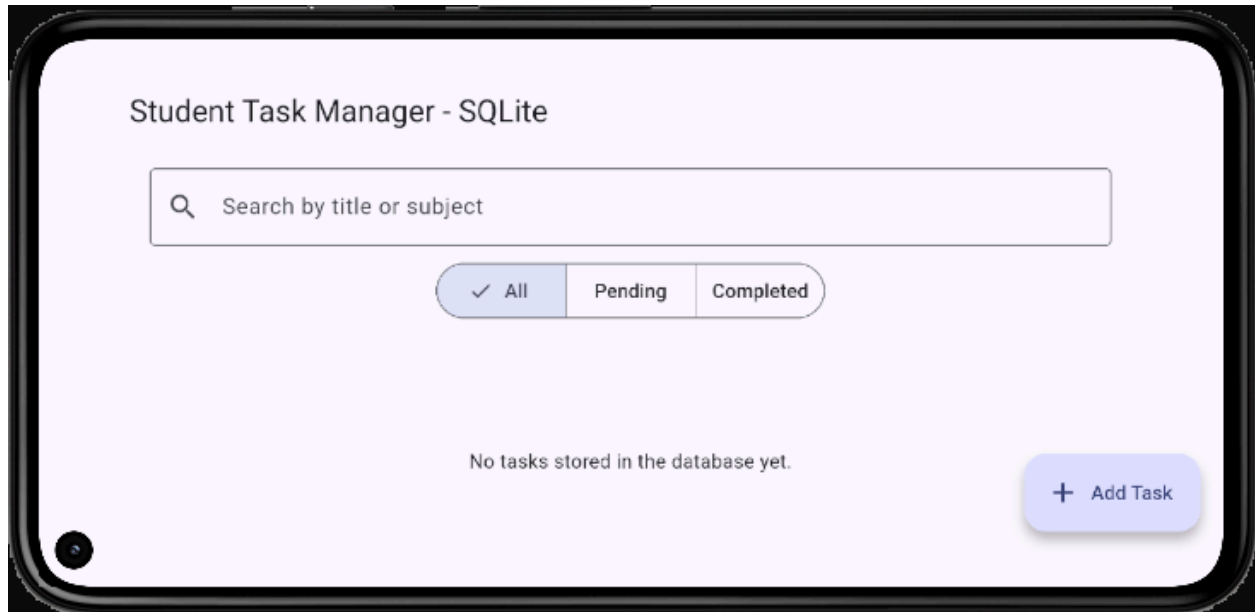
Close App → SQLite file remains → Reopen App → SELECT data → UI restored

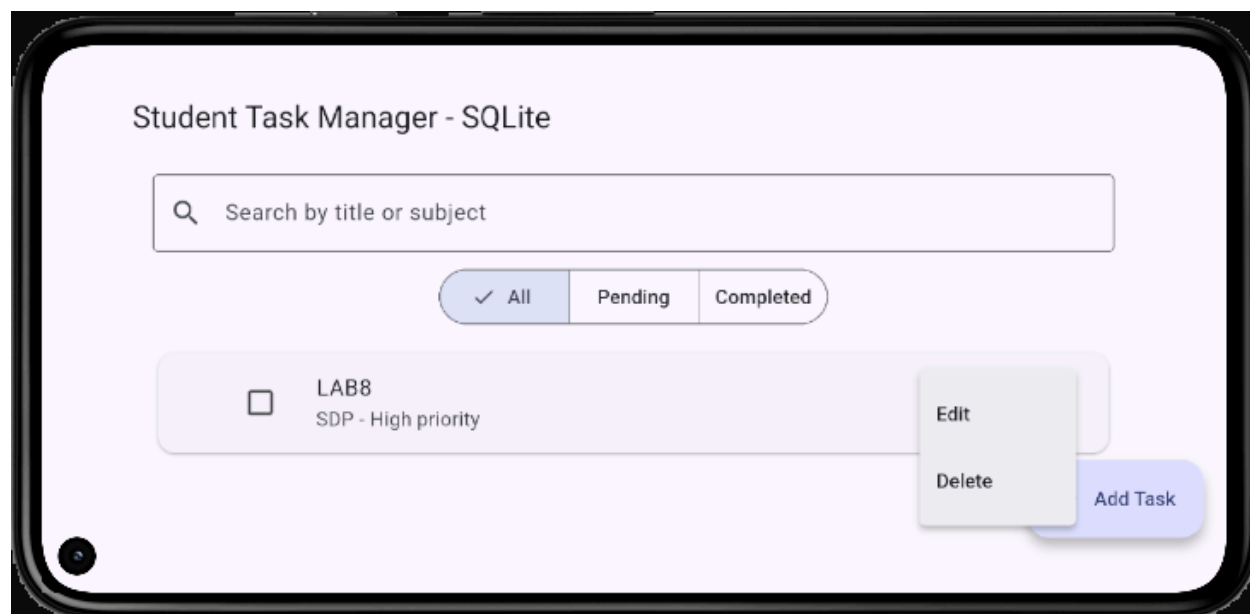
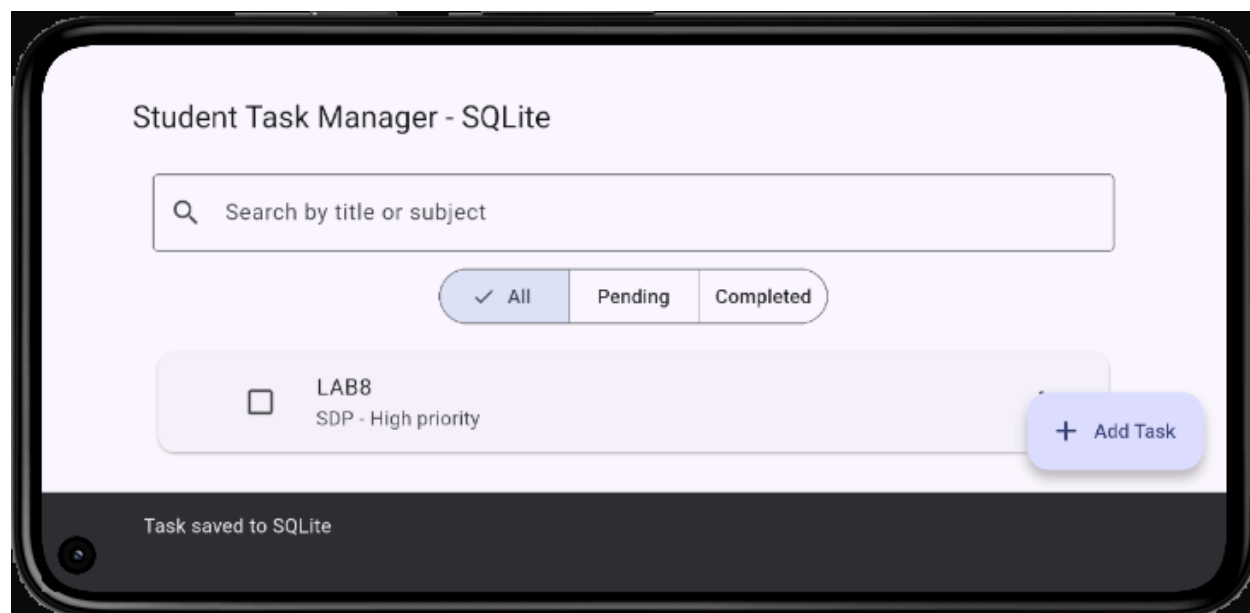
Database Schema / Table Definition

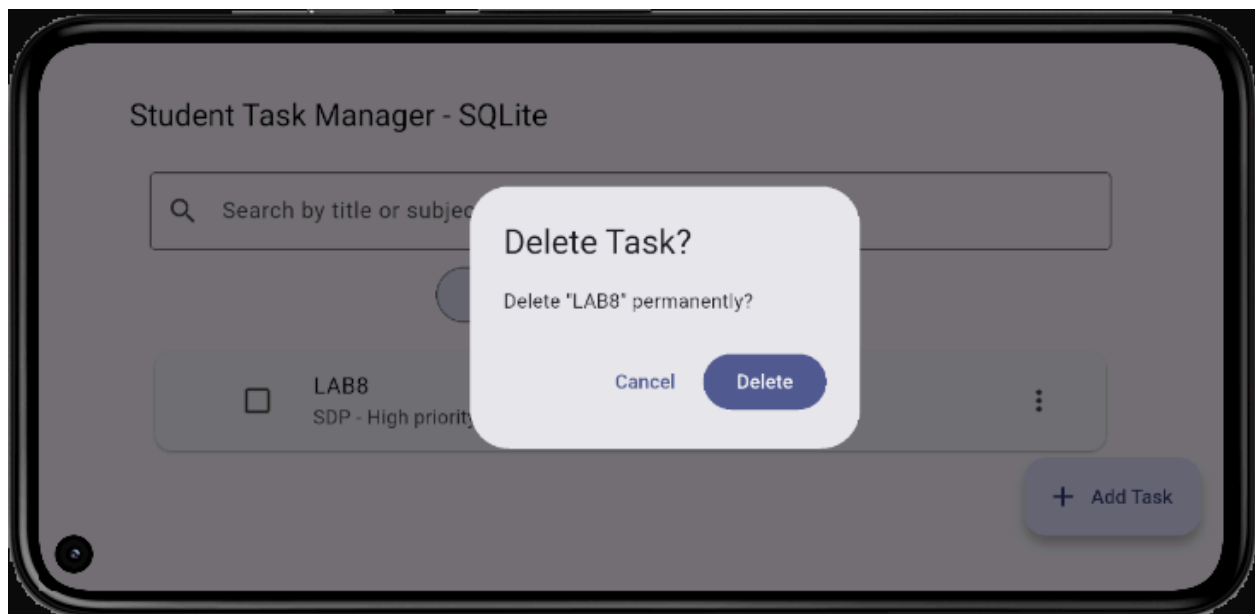
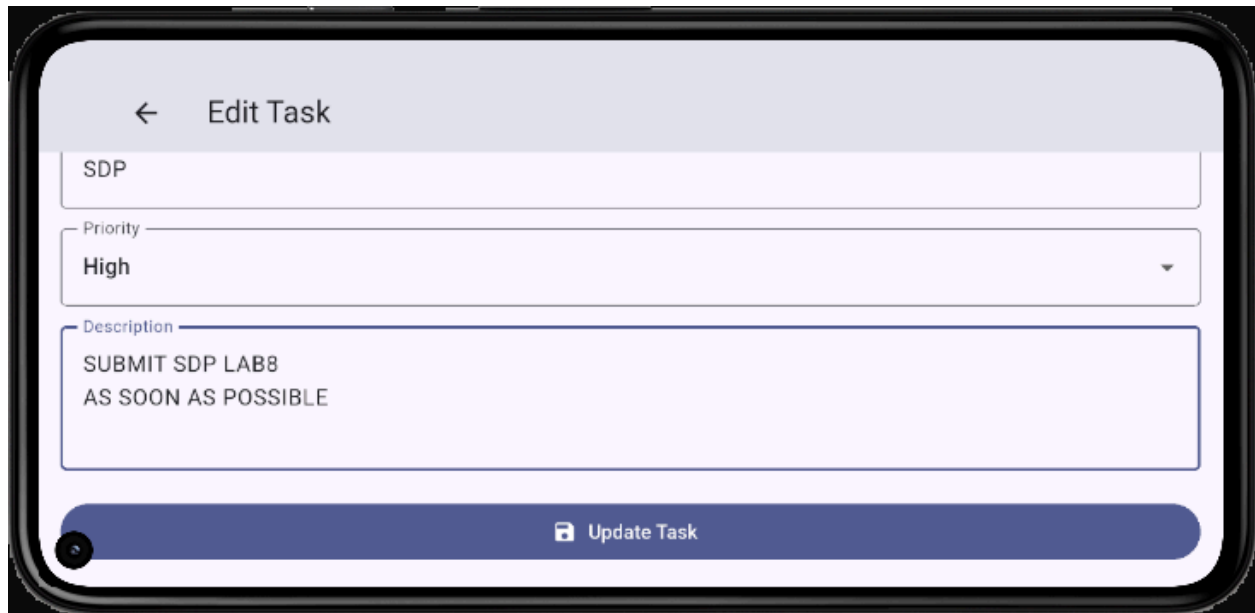
```
CREATE TABLE tasks(  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  title TEXT NOT NULL,  
  subject TEXT NOT NULL,  
  priority TEXT NOT NULL,  
  description TEXT NOT NULL DEFAULT '',  
  is_completed INTEGER NOT NULL DEFAULT 0  
);
```

Column	Type	Description
id	INTEGER	Unique ID generated automatically
title	TEXT	Task title
subject	TEXT	Subject of the task
priority	TEXT	Low, Medium or High
description	TEXT	Task description
is_completed	INTEGER	0 = false, 1 = true

INSERT / SELECT / UPDATE / DELETE







Model Mapping Explanation

Object → Map

The `toMap()` method converts a `Task` object into a map that can be stored in SQLite.

```

Map<String, Object?> toMap({bool includeId = true}) => {
  if (includeId && id != null) 'id': id,
  'title': title,
  'subject': subject,
  'priority': priority,
  'description': description,
  'is_completed': isCompleted ? 1 : 0,
};

```

Map → Object

The fromMap() method converts a database row back into a Task object.

```

factory Task.fromMap(Map<String, Object?> map) => Task(
  id: map['id'] as int,
  title: map['title'] as String,
  subject: map['subject'] as String,
  priority: map['priority'] as String,
  description: map['description'] as String,
  isCompleted: (map['is_completed'] as int) == 1,
);

```

Dart Task Object → toMap() → SQLite Row

SQLite Row → fromMap() → Dart Task Object

Important DatabaseHelper Methods

1. database

Opens the database when required and reuses the existing connection.

2. _initDatabase()

Creates the database path and opens the database.

3. _onCreate()

Creates the tasks table when the database is created for the first time.

4. insertTask()

Inserts a new task.

5. getTasks()

Retrieves all tasks.

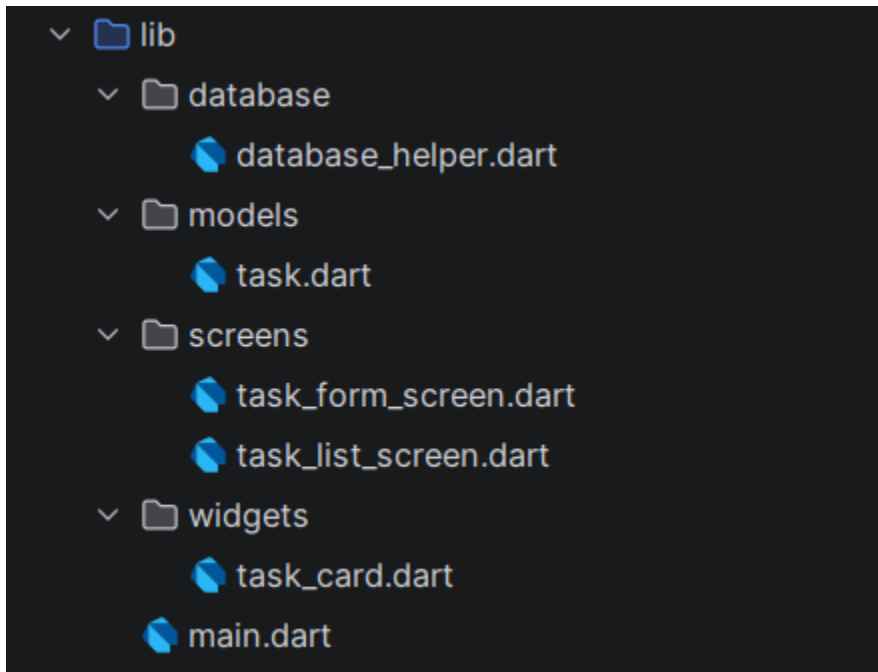
6. updateTask()

Updates an existing task using its ID.

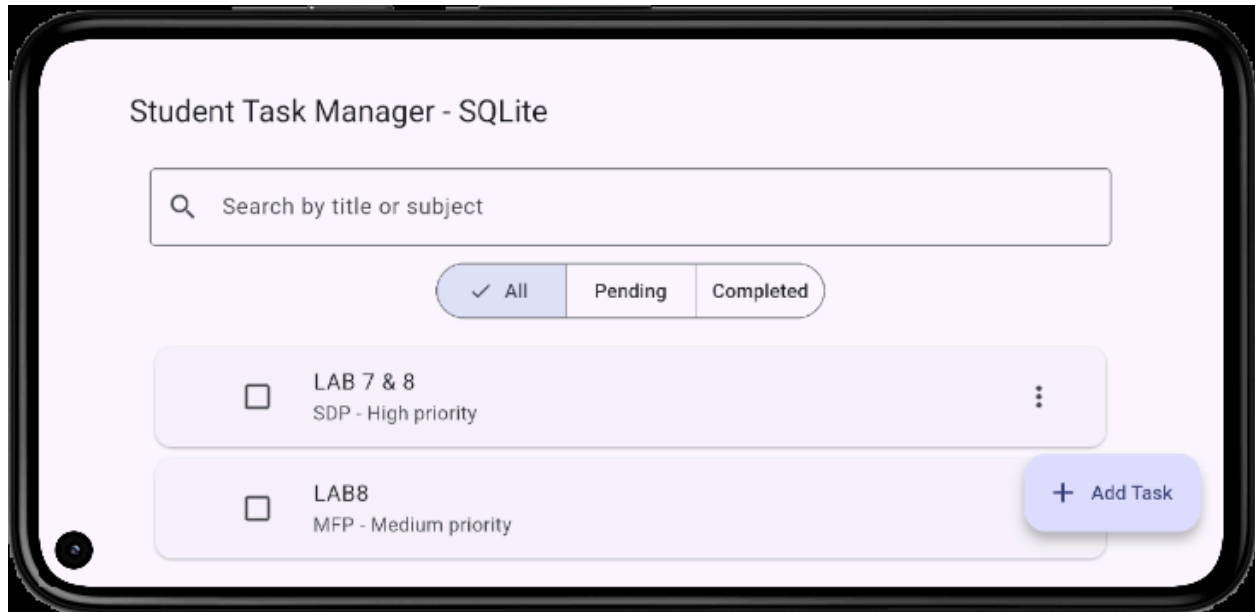
7. deleteTask()

Deletes a task using its ID.

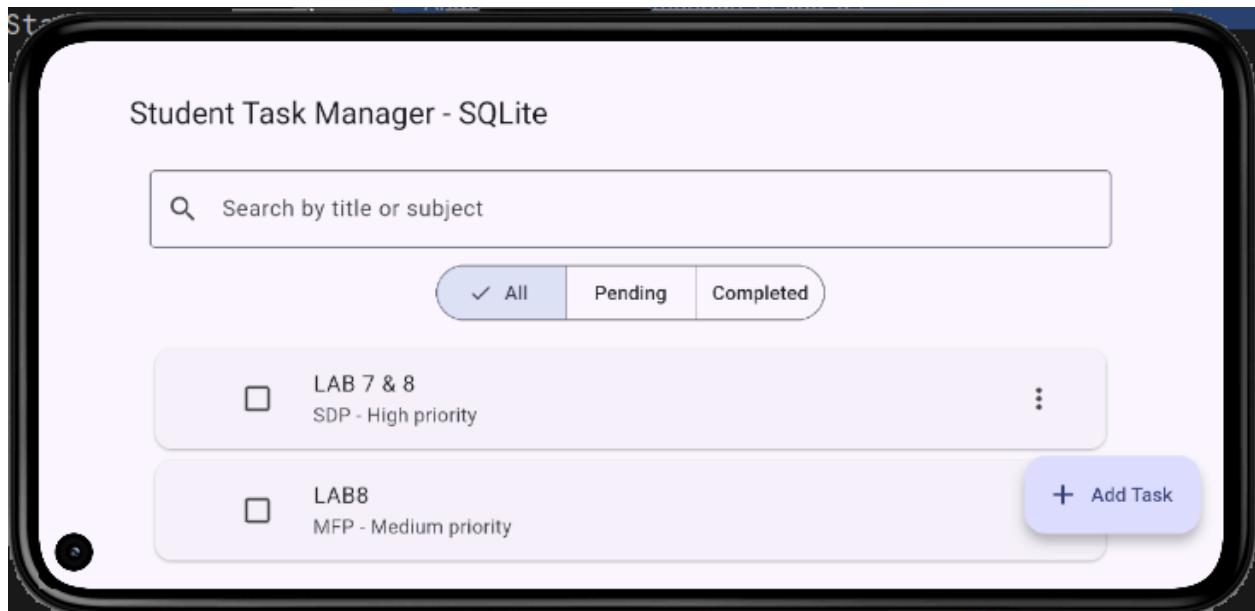
Project Structure



Proof of Persistence



After closing & reopening application:



The persistence of data was verified by adding tasks, completely closing the application, and reopening it. The previously stored tasks were still displayed after reopening, confirming that the data was successfully stored in SQLite.

Observation

The SQLite database successfully stored task data persistently. CRUD operations were performed using the sqflite package. The Task model was mapped to SQLite rows using

toMap() and fromMap(). After reopening the application, previously stored tasks were retrieved from the database and displayed again.

Result

The Flutter application was successfully integrated with SQLite for persistent data storage. INSERT, SELECT, UPDATE and DELETE operations were implemented successfully. The Task Manager was converted from temporary in-memory storage to persistent SQLite storage, and the stored tasks remained available after closing and reopening the application.
